# Morse Runner: macOS & Linux Porting Report

## Summary

This document describes the work done to port Morse Runner from its original Windows/Delphi codebase to macOS and Linux, resulting in a single merged repository (`MR_merge`) that supports all three platforms from one source tree.

| | Windows | macOS | Linux |
|---|---|---|---|
| **Compiler** | Delphi (RAD Studio) | Free Pascal (FPC 3.2.2+) | Free Pascal (FPC 3.2.2+) |
| **IDE/Build** | Delphi IDE | Lazarus `lazbuild` | Lazarus `lazbuild` |
| **GUI toolkit** | VCL (native Win32) | LCL + Cocoa widgetset | LCL + GTK2 widgetset |
| **Audio backend** | MMSystem (WaveOut API) | CoreAudio (AudioQueue) | PulseAudio (pa_simple) |
| **Data file paths** | `ParamStr(1)` (exe dir) | `.app/Contents/Resources/` | XDG (`~/.local/share/`) |
| **User file paths** | Exe directory | `~/Library/Application Support/MorseRunner/` | `$XDG_DATA_HOME/MorseRunner/` |
| **Project file** | `MorseRunner.dpr` / `.dproj` | `MorseRunner.lpi` | `MorseRunner_linux.lpi` |

## Porting Method: Directory Override, Not Ifdefs

Rather than scattering `{$ifdef WINDOWS}` / `{$ifdef DARWIN}` / `{$ifdef LINUX}` throughout every file, the port uses **separate files in platform-specific directories**, selected at compile time by the unit search path in each project file.

All three platforms compile the same shared contest logic (root `.pas` files). The platform-specific UI, audio, and file-path code lives in separate directories:

```
MR_merge/
├── *.pas                    ← Shared: contest logic, station, scoring
(32 files)
├── Util/                    ← Shared: parser utilities
├── VCL/                     ← Shared VCL units + Windows-specific _win
units
│
├── Main_win.pas             ← Windows: main form (VCL, Win32 API)
├── Log_win.pas              ← Windows: log display (TListView, TColor,
etc.)
├── Ini_win.pas              ← Windows: settings (IniFiles + Win32 types)
```

```
  │
  ├── mac/                        ← macOS + shared Unix/LCL code
  │   ├── Main.pas, Main.lfm    ← Main form (LCL, portable)
  │   ├── Log.pas                 ← Log display (TMemo-based, portable)
  │   ├── Ini.pas                 ← Settings (TIniFile, portable)
  │   ├── Globals.pas             ← Shared state (breaks circular deps)
  │   ├── ContestFactory.pas      ← Contest instantiation (breaks circular
deps)
  │   ├── PerlRegEx.pas           ← FPC RegExpr wrapper (replaces Delphi
TPerlRegEx)
  │   ├── GetDataPath.pas         ← .app bundle resource paths
  │   ├── ScoreDlg.lfm            ← Score dialog form
  │   └── VCL/
  │       ├── SndOut.pas          ← CoreAudio audio output
  │       ├── AudioBackend2.m     ← Objective-C CoreAudio ring-buffer backend
  │       ├── SndCustm.pas        ← Base audio class
  │       ├── WavFile.pas         ← Pure Pascal WAV I/O (replaces MMIO)
  │       ├── VolmSldr.pas        ← Volume slider (LCL Canvas)
  │       └── PermHint.pas        ← Hint window (LCLIntf)
  │
  ├── linux/                      ← Linux-only overrides (2 units)
  │   ├── GetDataPath.pas         ← XDG-compliant paths
  │   └── VCL/
  │       ├── SndOut.pas          ← PulseAudio audio output
  │       └── AudioBackendPulse.c  ← C PulseAudio ring-buffer backend
  │
  ├── MorseRunner.dpr             ← Windows project (Delphi)
  ├── MorseRunner.lpi             ← macOS project (Lazarus/Cocoa)
  ├── MorseRunner_linux.lpi       ← Linux project (Lazarus/GTK2)
  └── MorseRunner.lpr             ← Shared Lazarus program file (macOS +
Linux)
```

### How the search path selects the right files

Each `.lpi` project file sets a unit search path. When FPC encounters `uses SndOut`, it finds the first matching `SndOut.pas` on that path:

- **macOS**: `mac → mac/VCL → . → VCL → Util`
  - `SndOut` → `mac/VCL/SndOut.pas` (CoreAudio)
  - `GetDataPath` → `mac/GetDataPath.pas` (.app bundle)
- **Linux**: `linux → linux/VCL → mac → mac/VCL → . → VCL → Util`
  - `SndOut` → `linux/VCL/SndOut.pas` (PulseAudio)
  - `GetDataPath` → `linux/GetDataPath.pas` (XDG)
  - `Main` → `mac/Main.pas` (same LCL form — no linux override needed)
- **Windows**: Delphi `.dproj` explicitly names each unit file (`Main_win.pas`, `SndOut_win.pas`, etc.)

This approach means **zero shared files were modified for the port** (except 4 minor filename case fixes — see below). Adding macOS or Linux support does not touch any Windows code.

---

## What Was Changed vs. What Was Written New

### Shared files: zero functional changes

All 32 shared `.pas` files in the project root (`Contest.pas`, `Station.pas`, `DxOper.pas`, `CallLst.pas`, `ExchFields.pas`, etc.) compile on all three platforms **without modification**. They already used standard Pascal types and FPC-compatible constructs.

### 4 minor case-sensitivity fixes (affects Linux only)

Linux filesystems are case-sensitive; macOS and Windows are not. Four data file references had uppercase `.TXT` extensions that didn't match the actual lowercase `.txt` filenames on disk:

| File | Old reference | Fixed to |
|---|---|---|
| `CqWW.pas:76` | `'CQWWCW.TXT'` | `'CQWWCW.txt'` |
| `NaQp.pas:96` | `'NAQPCW.TXT'` | `'NAQPCW.txt'` |
| `ArrlSS.pas:100` | `'SSCW.TXT'` | `'SSCW.txt'` |
| `ArrlFd.pas:230` | `'FDGOTA.TXT'` | `'FDGOTA.txt'` |

These are the only edits to existing shared files. The fix is backwards-compatible — Windows and macOS are case-insensitive so lowercase works everywhere.

---

## New Files: macOS Port (mac/)

The macOS port required rewriting every unit that depended on Windows APIs. The `mac/` directory contains LCL/FPC equivalents:

### Main form: `mac/Main.pas` + `mac/Main.lfm` (2,687 + 1,499 lines)

Complete rewrite of `Main_win.pas` (2,761 lines). Replaces:

- All `Windows`, `Messages` unit references → LCL equivalents
- Win32 message handling (`WM_USER`, `WM_KEYDOWN`) → LCL `OnKeyDown`/`OnKeyUp` events
- `TToolBar`, `TImageList` (VCL) → `TPanel` + `TSpeedButton` (LCL-compatible)
- `System.ImageList`, `Vcl.ToolWin` → removed (not available in LCL)
- Timer-driven contest loop retained, same architecture

### Log display: `mac/Log.pas` (1,269 lines)

Rewrite of `Log_win.pas` (1,342 lines). Replaces:

- `System.UITypes.TColor` → local `TColor = Integer` typedef
- `TListView` score table → `TMemo`-based text display (LCL's `TListView` on macOS/Cocoa does not support per-cell coloring)
- `TStatusBar` panels → callback-based updates via `Globals.pas`

### Settings: `mac/Ini.pas` (416 lines)

Rewrite of `Ini_win.pas` (647 lines). Replaces:

- `TIniFile` usage is mostly the same (it's cross-platform)
- Removed Win32-specific default paths
- Added `GetUserPath` for platform-appropriate INI file location

### Globals: `mac/Globals.pas` (74 lines) — NEW

Does not exist in Windows version. Breaks circular unit dependencies that FPC 3.2.2 cannot resolve but Delphi handles silently. Holds:

- Debug flags (previously in `Main.pas`)
- Global string mirrors of exchange edit boxes
- Procedure-of-object callbacks for form actions
- References to key form controls (`TListView`, `TStatusBar`)

### ContestFactory: `mac/ContestFactory.pas` (48 lines) — NEW

Does not exist in Windows version. Extracted the contest-instantiation `case` statement from `Main.pas` into its own unit. This breaks the chain where `Main.pas` imported all 11 contest units in its `implementation` section, which caused FPC's unit resolver to fail.

### PerlRegEx: `mac/PerlRegEx.pas` (433 lines) — NEW

Wraps FPC's built-in `TRegExpr` to provide the `TPerlRegEx` API used by the Windows codebase (which uses a Delphi-specific PCRE binding). Only the methods actually called in this codebase are implemented.

### GetDataPath: `mac/GetDataPath.pas` (45 lines)

macOS-specific file paths:

- Read-only data: `.app/Contents/Resources/` (inside the app bundle)
- User-writable files: `~/Library/Application Support/MorseRunner/`

### Audio: `mac/VCL/SndOut.pas` (187 lines) + `AudioBackend2.m` (253 lines)

Replaces `VCL/SndOut_win.pas` (203 lines) + `VCL/SndCustm_win.pas` (267 lines).

Architecture:

```
Pascal main thread  → AudioBackend2_Write() → ring buffer (65536 frames)
CoreAudio callback  ← AudioQueueOutputCallback  ← pulls from ring buffer
                    → gNeedsData flag            → TTimer polls at 15 ms
```

CoreAudio uses a **pull model** (the OS calls our callback when it needs audio data), which naturally avoids buffer underrun issues.

`AudioBackend2.m` is Objective-C, compiled by Clang and linked via `{$L AudioBackend2.o}` in the Pascal source.

### Other VCL replacements

| Windows file | macOS replacement | What changed |
|---|---|---|
| `VCL/SndCustm_win.pas` | `mac/VCL/SndCustm.pas` | Removed Win32 MMSystem; kept TComponent base class |
| `VCL/WavFile_win.pas` | `mac/VCL/WavFile.pas` | Replaced Windows MMIO with pure Pascal TFileStream |
| `VCL/PermHint_win.pas` | `mac/VCL/PermHint.pas` | Replaced Win32 `CreateWindowEx` with LCLIntf calls |
| `VCL/VolmSldr_win.pas` | `mac/VCL/VolmSldr.pas` | Replaced `TGraphicControl.Canvas` with LCL Canvas |

## New Files: Linux Port (linux/)

The Linux port is much smaller because it **reuses all `mac/` files** except two units. The `mac/` files are LCL-generic — they use the Lazarus Component Library, not Cocoa-specific APIs. LCL works on both Cocoa (macOS) and GTK2 (Linux) without code changes.

### What Linux overrides (only 2 units)

**1. `linux/GetDataPath.pas` (44 lines)**

XDG Base Directory compliant paths:

- Read-only data: next to the binary (no `.app` bundle on Linux)
- User-writable files: `$XDG_DATA_HOME/MorseRunner/` (defaults to `~/.local/share/MorseRunner/`)

**2. `linux/VCL/SndOut.pas` (182 lines) + `AudioBackendPulse.c` (279 lines)**

PulseAudio backend, architecturally identical to the macOS CoreAudio backend:

```
Pascal main thread  → AudioBackendPulse_Write() → SPSC ring buffer (65536 frames)
Playback thread     → waits for >= 512 frames   → pa_simple_write() → PulseAudio
                    ← gNeedsData flag            ← TTimer polls at 15 ms
```

Key design decision: the playback thread **waits** when the ring buffer is empty instead of writing silence. PulseAudio is a push model (unlike CoreAudio's pull model), so injecting silence would buffer dead frames ahead of real audio, causing choppy playback.

### What Linux reuses from mac/ (no changes)

- `mac/Main.pas` + `mac/Main.lfm` — main form (LCL, works on GTK2 as-is)
- `mac/Log.pas` — TMemo log display
- `mac/Ini.pas` — settings
- `mac/Globals.pas` — shared state
- `mac/ContestFactory.pas` — contest instantiation
- `mac/PerlRegEx.pas` — regex wrapper
- `mac/VCL/WavFile.pas` — pure Pascal WAV I/O
- `mac/VCL/VolmSldr.pas` — volume slider
- `mac/VCL/PermHint.pas` — hint window
- `mac/VCL/SndCustm.pas` — base audio class (CoreAudio linkage is guarded by `{$ifdef DARWIN}`, safely skipped on Linux)

### Build scripts

- `build_linux.sh` — compiles `AudioBackendPulse.c` with gcc, then builds the Pascal project with `lazbuild --ws=gtk2`. Auto-detects x86_64 vs aarch64. Generates a `./MorseRunner` launcher wrapper that sets `GDK_NATIVE_WINDOWS=1` to avoid GTK2 depth-mismatch warnings under Wayland compositors.
- `install_deps.sh` — one-liner to install Ubuntu build dependencies (lazarus, fpc, libpulse-dev, pkg-config, gcc).

---

## Lines of Code Summary

| Category | Files | Lines |
|---|---|---|
| Shared contest logic (root `*.pas`, `Util/`) | 35 | ~12,000 |
| Shared VCL units (`VCL/`) | 10 | ~3,500 |
| Windows-only (`*_win.pas`, `VCL/*_win.pas`) | 6 | ~5,400 |
| macOS port (`mac/`, `mac/VCL/`) | 14 | ~7,000 |
| Linux port (`linux/`, `linux/VCL/`) | 3 | ~500 |
| Build scripts (Linux) | 2 | ~170 |

The Linux port is small (505 lines across 3 files) precisely because it reuses the macOS LCL code — only the audio backend and file paths differ.

---

## Key Engineering Decisions

### 1. Why not ifdefs?

The platform differences are too large. Windows `SndOut` uses Win32 MMSystem (`waveOutOpen`). macOS uses CoreAudio (`AudioQueueNewOutput`, Objective-C). Linux uses PulseAudio (`pa_simple_new`, C). These share an interface (`TAlSoundOut.PutData`, `.Enabled`, `.OnBufAvailable`) but zero implementation. Ifdefs would be unreadable. Separate files with the same unit name, selected by directory, is cleaner.

### 2. Why FPC + Lazarus, not Delphi cross-compile?

Delphi supports macOS and Linux cross-compilation, but:

- Requires an expensive Delphi Enterprise or Architect license
- FireMonkey (Delphi's cross-platform GUI) would require a full UI rewrite anyway
- FPC/Lazarus is free, open-source, and runs natively on all three platforms
- LCL provides native-looking widgets on each platform (Cocoa on Mac, GTK2 on Linux)

### 3. Why LCL GTK2, not GTK3 on Linux?

GTK3 with Lazarus LCL has two problems on Ubuntu:

- Forms designed for GTK2 metrics have oversized buttons and broken layouts
- LCL's GTK3 backend triggers `GtkCssCustomGadget` signal errors

GTK2 works correctly. The only cosmetic issue (depth-mismatch warnings under Wayland compositors) is solved by the `GDK_NATIVE_WINDOWS=1` environment variable in the launcher wrapper.

### 4. Breaking circular dependencies (Globals.pas, ContestFactory.pas)

Delphi silently resolves circular `implementation uses` dependencies. FPC 3.2.2 does not — it fails with "Can't compile unit." Two new units were created to break these cycles:

- `Globals.pas` — holds shared state that many units previously accessed via `Main.pas`
- `ContestFactory.pas` — holds the contest-instantiation case statement previously in `Main.pas`

These units do not affect the Windows build (Delphi ignores them).

### 5. Audio architecture consistency

All three platforms use the same high-level pattern:

1. A native audio backend manages a ring buffer and playback thread/callback
2. A `TTimer` on the LCL/VCL main thread polls a `NeedsData` flag at ~15 ms
3. When data is needed, `OnBufAvailable` fires and the app calls `PutData()`
4. `PutData()` writes normalized float samples to the ring buffer

This means `Main.pas` (and all contest logic) uses identical audio API calls on every platform — `AlSoundOut1.Enabled := true`, `AlSoundOut1.PutData(Tst.GetAudio)`.

---

## Testing Status

| Platform | Tested | Status |
|---|---|---|
| macOS (Apple M1, Sonoma) | Yes | Fully working — all contest modes, audio, keyboard, timer |
| Linux (Ubuntu 25.10, x86_64) | Yes | Fully working — all contest modes, PulseAudio audio |

| Platform | Tested | Status |
|---|---|---|
| Linux (Ubuntu, aarch64) | Build script supports it | Not yet tested on hardware |
| Windows (Delphi) | Not retested | No files changed — should be unaffected |

## How to Build

### Windows (unchanged)

Open `MorseRunner.dproj` in Delphi, build as usual.

### macOS

```
# Requires: Lazarus + FPC installed, Xcode command line tools
cd MR_merge
lazbuild MorseRunner.lpi --ws=cocoa
open MorseRunner.app
```

### Linux

```
# Requires: Ubuntu with lazarus, fpc, libpulse-dev, pkg-config, gcc
cd MR_merge
./install_deps.sh       # once
./build_linux.sh        # compile
./MorseRunner           # run
```